

tl;dr: An Evaluation Harness for LLMs on Code Tasks

What is llm-eval?

A rigorous evaluation framework for systematically benchmarking LLM performance on software engineering tasks.

Core Concepts:

- ▶ **Task:** An evaluation benchmark with dataset and metrics (e.g., code generation, test writing)
- ▶ **Experiment:** A configuration defining model, prompts, and parameters (temperature, runs, etc.)

Key Characteristics:

- ▶ **Statistically sound:** Bootstrap confidence intervals across multiple runs
- ▶ **Single-turn only:** No multi-turn agents (yet!)
- ▶ **Extensible:** Custom tasks, metrics, test runners, and providers
- ▶ **Gin-configured:** Flexible experiment definition and dependency injection

Core Capabilities

70+ Evaluation Metrics

- ▶ **Execution-based:** pass@k, compilation rates, coverage, etc.
- ▶ **Text similarity:** BLEU, ROUGE, ChrF, Edit distance, etc.
- ▶ **Task-specific:** Function calls, anonymization quality, etc.

20+ Software Engineering Tasks

- ▶ **Canonic code editing benchmarks:** HumanEval, MBPP, CanItEdit, etc.
- ▶ **Custom benchmarks:** Function calling, unit test generation, commit message generation, diff understanding, NL2Bash, etc.

10+ LLM Provider Support

- ▶ All major providers (OpenAI, Anthropic, Google, etc.)
- ▶ Local models via vLLM and Ollama

Architecture

Evaluation Pipeline:

- ▶ Dataset → Preprocessor → Prompt Template → LLM
- ▶ Response → Postprocessor → Test Runner (Docker)
- ▶ Results → Metrics & Statistical Analysis → Report

Two-layer caching system:

- ▶ Experiment-level (JSON, human-readable)
- ▶ Model-level (SQLite with diskcache)

Gin Configuration System

Simple, reproducible experiments with dependency injection:

```
1 # experiments/cmg/baseline.gin
2 Experiment.model_cls = @OpenaiApiModel
3 OpenaiApiModel.model_name = "gpt-4o"
4
5 Experiment.prompt = @ChatPromptTemplate()
6 ChatPromptTemplate.user = """Given the diffs,
7 generate a commit message
8 {diff}"""
9
10 Experiment.num_runs = 5
```

Run with one command:

```
1 python main.py --task=cmg \
2   --experiment=experiments/cmg/baseline.gin \
3   --use_coroutines --rpm_limit=100 \
4   --cache_level=full
```

Key Features

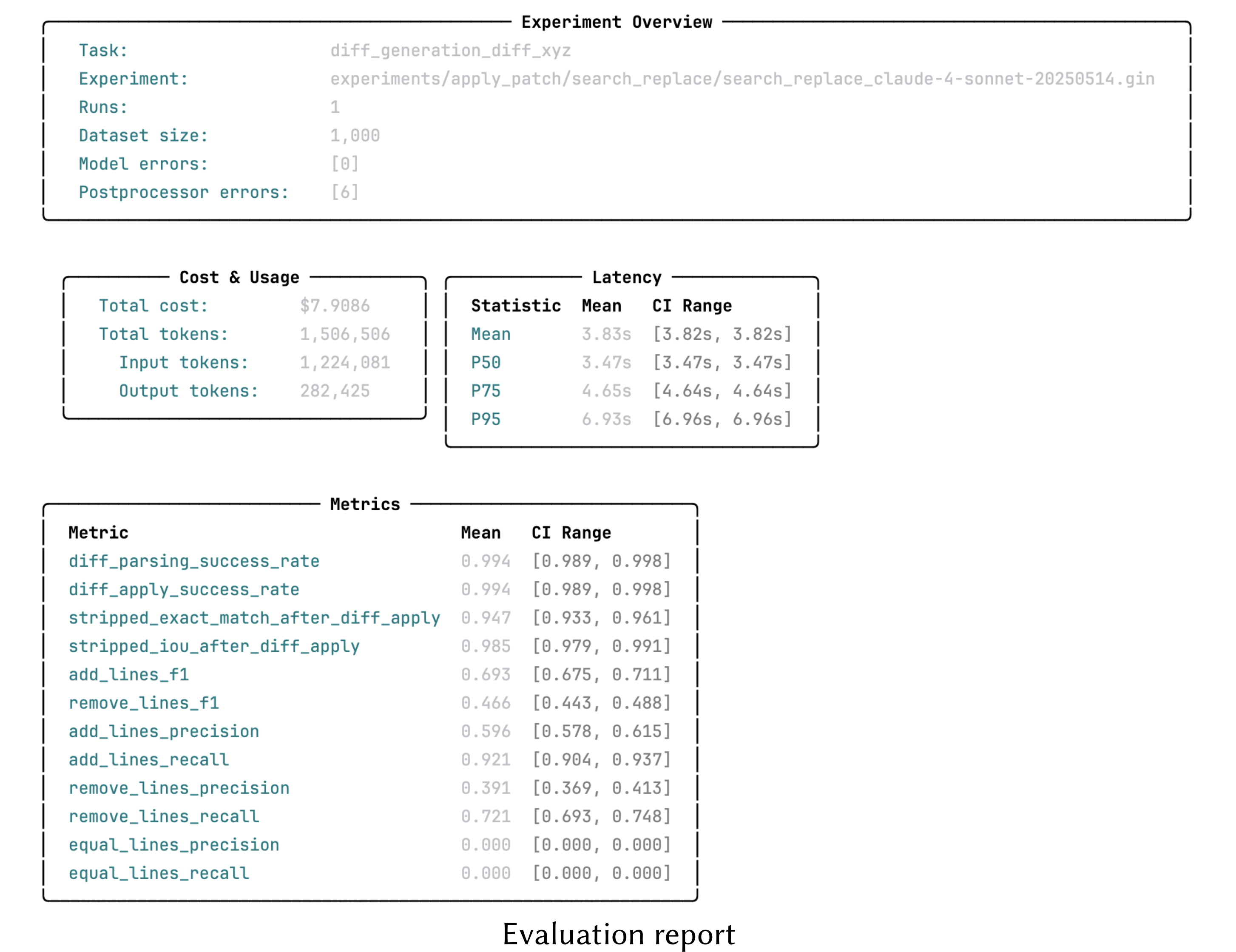
- Docker Sandboxing** Safe code execution for test runners
- Bootstrap CIs** 95% confidence intervals on all metrics
- Cost Tracking** Token usage and API cost estimation
- Latency Profiling** p50/p75/p95 percentiles

In Action

Actively used by our team for:

- ▶ **Fuzzy Apply:** Evaluating patch application accuracy
- ▶ **Edit Trajectory Synthesis:** Benchmarking code edit generation
- ▶ **Diff-XYZ:** Assessing diff understanding capabilities

Example Output



Repository

